

Kundai Farai Sachikonye
Auf der Lichtung 19, 82178 Puchheim
github.com/fullscreen-triangle
kundai.sachikonye@bitspark.com
(+49) 1626386344

17 May 2026

Ulrike Auer
EagleBurgmann Germany GmbH & Co. KG
Ulrike.Auer@eagleburgmann.com

Re: Scientific Innovator — AI-Driven Engineering (f/m/d)

Dear Ms Auer,

I am applying for the Scientific Innovator position. I have built original systems at each layer the role describes — agentic LLM workflows, surrogate models for physics-based simulation, and a formal training principle for domain-specific models — and I can demonstrate all of them running live in a browser before you read further:

- Multi-model RAG pipeline with hybrid LLM + mathematical solver selection: github.com/fullscreen-triangle/four-sided-triangle
- Agentic OS interface ($O(\log_k N)$ specialist cascade routing): github.com/fullscreen-triangle/zangalewa
- Domain-specific model training (Backward Training Principle): github.com/fullscreen-triangle/purpose
- First-principles gas and fluid dynamics (Navier-Stokes from partition theory, viscosity 2.9 % MAE): gas-giant.vercel.app/simulate

Surrogate models and the surrogate training problem. The central challenge in building surrogate models for FEM/CFD simulation is not the architecture — it is the training distribution. Conventional approaches build surrogates incrementally from less to more demanding operating conditions. My Purpose framework establishes the opposite as the formally optimal strategy: the **Backward Training Principle** proves that a model ϵ -competent on the hardest single-objective instance (the *pure-intent regime*) is ϵ -competent on every less-constrained sub-regime by *feasibility inclusion*, not by generalisation. For a seal, the pure-intent regime is the most extreme operating condition — highest pressure, temperature, and chemical exposure simultaneously — from which all normal operating envelopes are feasibility restrictions. Training the surrogate there produces a model that works across the full design space without re-training. A Forward Training Collapse Theorem establishes a provably positive VC failure-gap for the conventional direction; a Sample Complexity Theorem gives $(N + 1) \times$ savings for N -layer constraint cascades. I have already applied this pattern in practice: homo-veloce is a surrogate model system that distils physics-based biomechanics solvers (Gaussian processes and graph neural networks over physical simulation outputs) into lightweight neural networks for real-time inference — the same pattern FEM/CFD surrogate development requires.

CFD and fluid simulation from first principles. The position lists FEM/CFD simulation as one of the skill areas and surrogate modelling as the target application. The Gas Giant platform I have built takes a different approach: rather than training a surrogate on CFD data, it derives the governing equations from the same bounded phase space axiom. Kirchhoff circuit laws map analytically to the Navier-Stokes equations; viscosity emerges as $\mu = \tau_c \times g$ (partition lag times coupling strength) rather than as a phenomenological fit. The gas-to-liquid transition occurs at network density thresholds $\rho_C \approx 0.3\text{--}0.7$, with no fitted transition parameter. Validation: 2.9 % mean absolute error across 12 pure liquids for viscosity; 0.9 % mean error for Poiseuille flow parabolic velocity profiles. Temperature emerges as $T = (\hbar/k_B) \cdot dM/dt$ — a processing rate identity — from which the ideal gas law $PV = Nk_B T$ and the adiabatic invariant $PV^\gamma = \text{const}$ follow as corollaries, validated to $U/(32Nk_B T) = 1.000 \pm 0.2\%$. For sealing technology specifically, the fluid dynamics around the seal face — viscosity, diffusion, pressure gradients, flow regime transitions — are exactly what the /fluids tool computes, from partition geometry rather than from empirical correlations.

Digital twin and condition monitoring. The Syndrome framework encodes system state as a vector $\mathbf{D} \in [0, 1]^8$ across eight monitoring dimensions, each with a coherence index $\eta_i \in [0, 1]$ computed from sensor time-series regularity. The framework computes $\mathcal{O}(N \log N)$ state trajectories and selects the minimum-intervention maintenance action via sparse ℓ_1 linear programming. This architecture was developed for biological systems but the formalism is domain-agnostic: the eight oscillator classes map directly onto the relevant condition monitoring dimensions for rotating or dynamic sealing equipment (leakage rate, vibration spectrum, temperature distribution, pressure differential, wear pattern, corrosion state, fatigue index, and interfacial geometry). The sparse LP answers the predictive maintenance question directly: which component to address, when, and in what order, to minimise downtime and avoid cascading failure. The Verum framework adds a complementary state-reconstruction layer: a vehicle modelled as a 20-node oscillatory circuit graph, where each subsystem’s state is its categorical depth $V_i = -\log_2 P(\sigma_i)$ and edge conductances derive from a universal transport formula unifying electrical, thermal, viscous, and diffusive coupling. A Banach contraction mapping (Lipschitz constant $\lambda \in [0.3, 0.7]$) guarantees reconstruction of the complete state from 33 % surface observations alone — validated on 2023 Formula One telemetry (Bahrain GP, 13,794 samples, convergence residual 7.2×10^{-9} , ICE-Turbo fault detected 13 laps before failure event). The same architecture maps directly onto sealing equipment: the circuit graph nodes are pressure differential, temperature, vibration, wear state, and interfacial geometry; the surface sensors are the observed boundary conditions; the complete condition monitoring vector is the hidden state recovered by the contraction.

Agentic workflows and LLM engineering assistants. Four-Sided-Triangle is an 8-stage metacognitive RAG pipeline with adaptive hybrid selection between LLM reasoning and mathematical solvers, backed by a Rust engine providing 10–50× speedup over Python for compute-intensive stages. Zangalewa is the Minimum Sufficient Interceptor sitting at the boundary of a blank-screen OS: a user types a natural language request; the system extracts semantic coordinates via a hierarchical k -ary specialist cascade requiring $\mathcal{O}(\log_k N)$ small-model invocations, and routes to the correct tool or expert sub-model. A Sufficiency Theorem proves the parameter budget is $\mathcal{O}(D \cdot b \cdot \log |\Sigma|)$, independent of the downstream tool space size. For an LLM engineering assistant that must route between FEM queries, material databases, maintenance logs, and design automation tools, this is the architecture

that keeps the routing layer small and the specialist tools deep.

Distributed sensor coordination. A digital twin for sealing technology aggregates sensor streams from multiple physical sites under real-time timing and security constraints. The Pylon framework I have developed addresses this directly: it proves a structural identity between distributed networks and ideal gases ($PV = Nk_B T$, validated to 0.6% on a 128-node, 10^6 -packet testbed) and derives coordination protocols from that identity, achieving $33\times$ throughput improvement, $20\times$ jitter reduction, and $1000\times$ faster recovery versus conventional approaches. Security derives from the Second Law of Thermodynamics rather than cryptographic hardness: any coordinated sensor spoofing attack requires a thermodynamically forbidden entropy reduction, giving 100% detection with 0% false positives. The precision-by-difference synchronisation layer ($\Delta P(v_i, k) = T_{\text{ref}}(k) - t_i(k)$ against an atomic clock reference) provides 87 ns phase coherence across nodes — the timing substrate that makes the condition-monitoring state vector $\mathbf{D}$ in the Syndrome framework trustworthy at the sensor level.

Background. I hold an MSc in Computational Biology (Universität Tübingen), an MSc in Pharmaceutical Biotechnology (HAW Hamburg), and completed doctoral research in computational science at TUM. My primary languages are Python (PyTorch, scikit-learn, Ray), Rust for performance-critical components, and TypeScript for deployment. I am legally resident in Germany with full work authorisation, available immediately, and the Wolfratshausen location is a short commute from where I currently live. English is native; German is conversational.

Yours sincerely,

Kundai Farai Sachikonye